Embedded Systems (M233)
The Complete Offline Guide

Lectures 1–2 and all labs, from A to Z: the Arduino Uno, C++ on a microcontroller, digital I/O, timing with `millis()`, Serial, finite state machines. With 27 auto-checked exercises, every lab solved, and a simulator that grades your code without internet.

Compiled from the course of Pr. Aissam Berrahou (UM6P College of Computing), extended and verified

Every sketch in this guide was compiled and run in the included simulator and syntax-checked with `avr-g++` for the ATmega328P.

Summary: How to use this kit on the plane

The folder `embedded-systems/kit` is self-contained (Python 3 and the Xcode command-line tools are already on this Mac; no internet needed).

```
cd ~/textbooks/embedded-systems/kit      # (or ~/study/embedded/kit)
python3 check.py list                    # the 27 exercises
cp exercises/ex06_two_timers/starter.ino exercises/ex06_two_timers/sketch.ino
python3 arduino_sim.py run exercises/ex06_two_timers/sketch.ino --wave   # timeline
python3 arduino_sim.py run x.ino --press 1000:3 --press 2500:3 --wave    # button on pin 3
python3 arduino_sim.py run x.ino --serial "1000:ON\n"                    # text from the PC
python3 check.py ex06                    # grade it: PASS or the first failures
python3 check.py all --solution          # re-verify every reference solution
python3 labs/verify_labs.py              # re-verify every lab solution
python3 arduino_sim.py avr x.ino         # compile for the real ATmega328P (int = 16 bits!)
```

Read the parts in order (A to G, about 20 pages), doing the exercises as they are referenced; then use parts H–J as the answer key and reference.

Contents

1 A. Embedded Systems and the Arduino Uno **2**

2 B. C++ on a Microcontroller: the Refresher **3**

2.1 The sketch and the hidden `main()` . . . 3

2.2 Types: sizes on the AVR . . . 4

2.3 Scope, lifetime, constants . . . 4

2.4 Control flow and the two `switch` traps . . . 4

2.5 Arrays, `sizeof`, functions, references, pointers . . . 5

2.6 `struct`, `enum`, `enum class` . . . 5

2.7 Memory and the things not to do . . . 5

3 C. Digital Output: LEDs, Current, PWM, Registers **6**

4 D. Digital Input: Floating Pins, Pull Resistors, Bouncing **7**

4.1 Bouncing and the debounce algorithm 8

5 E. Time: `delay()` versus `millis()` **8**

6 F. Serial Communication (UART) **9**

6.1 Arduino to Arduino 11

7 G. Finite State Machines **11**

7.1 Three ways to write the same machine 12

7.2 Timed states without `delay()` 13

8 H. The Labs, Solved **14**

8.1 Lab: the on-board LED (pin 13) 14

8.2 Lab: blinking an external LED 15

8.3 Lab: Serial communication 15

8.4 Lab: the floating input and pull resistors 16

8.5 Lab: `millis()` and the bouncing problem 16

8.6 Lab: traffic lights with an FSM 17

8.7 Lecture-2 labs and Canvas assignments 17

9 I. The 27 Exercises and Their Solutions **18**

10 J. One-Page Reference **33**

1 A. Embedded Systems and the Arduino Uno

Concept: Embedded system

An **embedded system** is an autonomous electronic and computing system dedicated to one or a few functions, integrated into a larger product, and usually subject to tight constraints on CPU, memory, power and *time*. Four families are distinguished by how they operate and communicate:

Type	What defines it	Examples
Stand-alone	works without a host computer or network	washing machine, microwave
Real-time	must answer events within a deadline	ABS, airbag controller, ADAS
Networked	exchanges data over a network	smart meter, IoT sensor node
Mobile	battery powered, very limited resources	smartwatch, GPS tracker

Development boards. Arduino UNO (education, sensors, simple automation; this course), ESP32 (Wi-Fi + Bluetooth, IoT), STM32 (ARM Cortex-M, industrial control, robotics), Raspberry Pi (a Linux single-board computer, not a microcontroller). "Arduino" names three things: the IDE on the PC, the board, and the firmware (bootloader + core library) on the chip.

Concept: The Arduino Uno in numbers

Microcontroller	ATmega328P, 8-bit AVR RISC	Flash	32 KB, non-volatile: the program (0.5 KB bootloader)
Clock	16 MHz (62.5 ns per cycle)	SRAM	**2 KB**, volatile: globals, stack, heap
Logic voltage	5 V (chip: 1.8–5.5 V)	EEPROM	1 KB, non-volatile: settings, counters; slow write (≈3.3 ms/byte)
Digital pins	D0–D13; D0/D1 = Serial RX/TX	PWM pins	3, 5, 6, 9, 10, 11 (marked ~)
Analog inputs	A0–A5, 10-bit ADC (0–1023)	Pin current	40 mA absolute max, 20 mA recommended

2 KB of SRAM is the number to remember: a 100-character `String`, a 500-element `int` array (1000 bytes) or deep recursion exhausts it with no error message, only strange behaviour.

Workflow. Write the sketch in the IDE (or Tinkercad/Wokwi online, SimulIDE/Proteus offline) → *Verify* compiles it with `avr-g++` into a `.hex` file → *Upload* resets the board, the bootloader receives the hex over USB-serial and writes it to flash → the program runs from `main()` forever. On the Uno the USB chip appears as `COMx` (Windows) or `/dev/ttyACM0` (Linux); Tools → Board → Arduino Uno, Tools → Port.

2 B. C++ on a Microcontroller: the Refresher

You know C++. What changes on an 8-bit AVR is the *sizes*, the *absence of an operating system*, and the discipline that 2 KB of RAM imposes.

2.1 The sketch and the hidden `main()`

```
// 1. constants and globals
const int LED_PIN = 13;
// 2. runs once
void setup() {
  pinMode(LED_PIN, OUTPUT);
  Serial.begin(9600);
}
// 3. runs forever
void loop() {
  digitalWrite(LED_PIN, HIGH); delay(500);
  digitalWrite(LED_PIN, LOW);  delay(500);
}
```

```
// what the Arduino core really compiles:
int main() {
  init();          // timers for millis(), ADC, ...
  setup();         // yours, once
  while (1) {
    loop();        // yours, forever
    // (serialEventRun() between calls)
  }
}
```

An embedded program never returns: there is nothing to return to.

Warning / Gotcha: Prototypes are generated for you, and that has a trap

In C++ a function must be declared before use. The Arduino builder scans the sketch and inserts prototypes for every function at the top of the file, so `setup()` may call `blink()` defined further

down. Tinkercad inserts them *before* your type definitions, so a function whose parameter is one of your own types (`void processEvent(Event e)`) produces "`Event` was not declared". The lab's work-around is a default argument, `void processEvent(Event e = Event::TIMEOUT)`, which the scanner skips. Writing the prototype yourself after the `enum` also works. The kit's simulator reproduces both behaviours.

2.2 Types: sizes on the AVR

Type	Size	Range	Use
`bool` / `boolean`	1 byte	true/false	flags, states
`byte` = `uint8_t`	1 byte	0..255	pin numbers, bytes on the wire
`char`	1 byte	−128..127	one ASCII character (`'A'` is 65)
`int`	**2 bytes**	−32 768..32 767	general counters: **overflows at 32 767**
`unsigned int` = `uint16_t`	2 bytes	0..65 535	
`long`	4 bytes	$\pm 2.1 \times 10^9$	
`unsigned long` = `uint32_t`	4 bytes	0..4 294 967 295	**everything returned by `millis()`**
`float` = `double`	4 bytes	$\pm 3.4 \times 10^{38}$	slow (software emulation); avoid in loops
`String`	heap		convenient, but fragments the 2 KB heap

Warning / Gotcha: Three bugs that only exist on the AVR

- `int ms = 40000;` is −25 536. `int x = 1000 * 60;` overflows. Use `long` or `unsigned long` for anything above 32 767, and `UL` suffixes in constant expressions: `60UL * 1000`.
- Storing `millis()` in an `int` wraps after 32 s. `unsigned long now = millis();` always.
- `10 / 4` is 2 (integer division); `10 / 4.0` is 2.5. Also `7 % 3` is 1.

On the host (your Mac) `int` is 32 bits, so these bugs *disappear* in the simulator. That is why the kit also offers `python3 arduino_sim.py avr x.ino`, which type-checks with the real cross-compiler.

2.3 Scope, lifetime, constants

```
int globalCounter = 0;              // global: everywhere, lives forever (in SRAM)
void loop() {
  int local = 5;                    // local: created at each call, on the stack
  static int calls = 0; calls++;    // static local: initialised once, keeps its value between calls
}
#define LED_PIN 13                  // preprocessor text replacement: no type, no scope, no debugger
    name
const int BUTTON_PIN = 2;           // typed constant (preferred); the compiler folds it, no RAM used
constexpr int MAX_COUNT = 100;      // compile-time constant (C++11), same idea, usable in constant
    expressions
```

2.4 Control flow and the two `switch` traps

`if` / `else if` / `else`, `for`, `while`, `do ...while` are standard. Always use braces. `switch` compares one integer or enum against constant cases: cleaner than long `else if` chains and the natural shape of a state machine.

```
switch (currentState) {
  case State::IDLE:                         // trap 1: forgetting break falls through into the next
    case
```

```
    if (pressed) currentState = State::ACTIVE;
    break;
  case State::BLINK: {                        // trap 2: a variable declared under a case label needs
    its own { } block,
    unsigned long now = millis();             //         otherwise "jump to case label crosses
    initialization"
    if (now - t0 >= 500) toggle();
    break;
  }
  default: break;
}
```

A `while (!Serial.available());` busy-wait is acceptable in `setup()`, never in `loop()`: it blocks everything else (see §5).

2.5 Arrays, `sizeof`, functions, references, pointers

```
const int LEDS[] = {9, 10, 11};
const int NB_LEDS = sizeof(LEDS) / sizeof(LEDS[0]);   // 3, computed by the compiler (sizeof in bytes:
    6 / 2)
for (int i = 0; i < NB_LEDS; i++) digitalWrite(LEDS[i], HIGH);

void increment(int x)  { x++; }    // by value: a copy, caller unchanged
void realInc(int &x)   { x++; }    // by reference: modifies the caller's variable
int value = 42; int *p = &value;   // pointer: p holds the address of value
*p = 100;                          // dereference: value is now 100
void (*action)() = &turnOn;        // pointer to a function taking nothing, returning nothing
action();                          // calls turnOn()  (used by the table-driven FSM, part G)
```

2.6 `struct`, `enum`, `enum class`

```
struct Button {              // group related data
  uint8_t pin;
  bool currentState;
  bool previousState;
  unsigned long lastDebounce;
};
Button buttons[] = {         // array of structs
  {2, HIGH, HIGH, 0},
  {3, HIGH, HIGH, 0},
};
buttons[0].pin;              // field access
```

```
enum Light { RED, YELLOW, GREEN };      // plain
    : RED is an int (0),
Light l = RED; int x = l + 1;           // mixes
     silently with ints

enum class State : uint8_t { OFF, ON }; //
    scoped + typed:
State s = State::OFF;                   // must
    be qualified,
// int y = s;      // error: no implicit int
// if (s == 0)     // error: compare with State
    ::...
```

`enum class` is what the labs use for states and events: the compiler refuses to confuse a `State` with an `Event` or an `int`.

2.7 Memory and the things not to do

Flash holds code and `const` data; SRAM holds globals and statics (from address 0x100 upward), the heap (`malloc`, `String`) growing up and the stack growing down. When they meet, variables get corrupted silently. Rules: prefer `const` and small types, avoid `String` concatenation in `loop()`, use `F("text")`

to keep string literals in flash when printing, keep arrays small, and avoid recursion. `volatile` marks variables shared with interrupt handlers so the compiler re-reads them each time (not needed in this course's labs, which use no interrupts).

3 C. Digital Output: LEDs, Current, PWM, Registers

Interface / API: The three calls

```
pinMode(pin, OUTPUT);          // in setup(): configure the pin as a push-pull driver (also INPUT,
    INPUT_PULLUP)
digitalWrite(pin, HIGH);       // 5 V on the pin   (HIGH = 1)
digitalWrite(pin, LOW);        // 0 V on the pin   (LOW = 0)
analogWrite(pin, 0..255);      // PWM on pins 3,5,6,9,10,11 only: a square wave whose duty cycle is
     value/255
```

Pin 13 and the "L" LED. The Uno's on-board LED is wired to header pin D13, which is port pin **PB5** of the ATmega328P, through a 1 kΩ series resistor. `LED_BUILTIN` is a constant equal to 13 on the Uno and to the right pin on other boards; use it for portability.

Method: Sizing the series resistor (Ohm's law)

An LED is a diode: it conducts only from anode (+, long leg) to cathode (−, short leg, flat edge of the body), and once conducting it has almost no resistance. Without a resistor the current is limited only by the pin's driver: the LED or the pin burns (Tinkercad shows smoke). With $V_{CC} = 5$ V and a forward voltage $V_F \approx 2$ V (green/red; blue/white ≈ 3 V):

$$I_{LED} = \frac{V_{CC} - V_F}{R}$$

$R = 1$ kΩ	$I = 3/1000$	$= 3$ mA (on-board LED, dim)
$R = 330$ Ω	$I = 3/330$	$= 9.1$ mA (safe)
$R = 220$ Ω	$I = 3/220$	$= 13.6$ mA (bright, safe)
$I = 20$ mA wanted	$R = 3/0.020$	$= 150$ Ω (an E12 value)

Limit: 40 mA absolute maximum per pin, 20 mA recommended, 200 mA for the whole chip. Wiring: pin → resistor → anode; cathode → GND. The resistor can equally go on the cathode side: what matters is that it is in series.

Period, frequency, duty cycle. For a blinking signal $T = t_{ON} + t_{OFF}$, $f = 1/T$, duty cycle $D = t_{ON}/T$. `delay(1000)` twice gives $T = 2$ s, $f = 0.5$ Hz; 250 ms twice gives $T = 0.5$ s, $f = 2$ Hz; 100/900 gives $f = 1$ Hz with $D = 10\%$. Above about 50 Hz the eye no longer sees flicker (persistence of vision): a 100/100 ms blink is still visible, 5 Hz.

PWM. `analogWrite(pin, d)` does not output an analog voltage: it starts a hardware timer that toggles the pin at ≈ 490 Hz (980 Hz on pins 5 and 6) with the pin HIGH for a fraction $d/255$ of each period. The LED is switched on and off 490 times a second and the eye averages the brightness. On a pin without a timer channel (pin 4, for instance) `analogWrite` degrades to `digitalWrite(value < 128 ? LOW : HIGH)`. Exercise ex05.

Mechanism: What `pinMode` and `digitalWrite` do: the port registers

The chip groups pins into 8-bit ports. Each port has three registers: `DDRx` (data direction: 1 = output), `PORTx` (level driven when output; pull-up enable when input), `PINx` (the physical level, read-only).

Arduino pin	Port bit	Example
D0–D7	PD0–PD7	pin 2 = PD2
D8–D13	PB0–PB5	pin 13 = PB5
A0–A5	PC0–PC5	A0 = PC0

```
DDRB  |=  (1 << PB5);   // pinMode(13,
    OUTPUT)
PORTB |=  (1 << PB5);   // digitalWrite(13,
    HIGH)
PORTB &= ~(1 << PB5);   // digitalWrite(13,
    LOW)
PORTB ^=  (1 << PB5);   // toggle
bool b = PIND & (1 << PD2);   // digitalRead
    (2)
```

A register write takes one clock cycle and can set 8 pins at once; `digitalWrite` takes about 50 cycles because it translates the pin number at run time. Exercise ex26.

4 D. Digital Input: Floating Pins, Pull Resistors, Bouncing

`digitalRead(pin)` returns HIGH if the pin voltage is above about 2.5 V (actually $> 0.6V_{CC}$), LOW if below ($< 0.3V_{CC}$), undefined in between. The question is always: *what sets the voltage when the button is not pressed?*

Warning / Gotcha: A floating input

A pin configured as `INPUT` with a button wired to GND and nothing else is connected to *nothing* when the button is released. Its input impedance is about 100 MΩ, so picoamps of leakage, your hand's capacitance, or 50 Hz mains hum move it across the 2.5 V threshold at random: "Button Pressed" / "Released" flicker, LEDs blink by themselves, counts increment with nobody touching anything. Simulators do not model this (they return a steady value), which is why the lab says it must be seen on real hardware. The kit's simulator returns random noise for a floating read and prints a warning.

Concept: The three stable wirings

Wiring	Circuit	`pinMode`	Reading
External **pull-down**	resistor (10 kΩ) from pin to GND; button from pin to 5 V	`INPUT`	released LOW, **pressed HIGH** (active-high)
External **pull-up**	resistor (10 kΩ) from pin to 5 V; button from pin to GND	`INPUT`	released HIGH, **pressed LOW** (active-low)
Internal pull-up	button from pin to GND; the chip's 20–50 kΩ resistor to 5 V is switched on by software	`INPUT_PULLUP`	released HIGH, **pressed LOW**

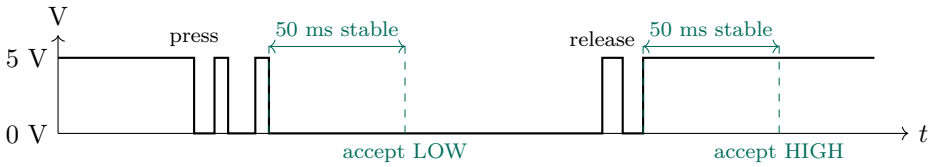
The resistor is what makes the level *defined* when the switch is open; when the switch is closed it limits the current from 5 V to GND to 0.5 mA instead of a short circuit. `INPUT_PULLUP` is preferred in low-cost projects: no extra part, no extra wire, and a button to GND is the easiest to wire. The price is inverted logic: write `bool pressed = (digitalRead(BUTTON) == LOW);` once and think in terms of `pressed` afterwards.

Exercises ex09 (follow), and the three-button lab (one wiring of each kind): `labs/07_slide_labs/`

three_buttons.ino.

4.1 Bouncing and the debounce algorithm

A mechanical contact does not close cleanly: for 5–20 ms it bounces, producing a burst of HIGH/LOW transitions. A sketch that counts falling edges sees one press as 2–5 presses. `delay(50)` after a change "works" but freezes the CPU; the course's algorithm uses `millis()`:



Method: Debounce with `millis()` (the lecture's function, returns true once per press)

```
const unsigned long DEBOUNCE_DELAY = 50;
int buttonState = HIGH;            // last ACCEPTED (stable) state; HIGH = released with a pull-up
int lastReading = HIGH;            // last RAW reading
unsigned long lastDebounceTime = 0;

bool buttonPressed(int pin) {
  int reading = digitalRead(pin);
  bool pressed = false;
  if (reading != lastReading) lastDebounceTime = millis();      // any change, real or bounce,
    restarts the timer
  if (millis() - lastDebounceTime > DEBOUNCE_DELAY) {             // the reading has been stable
    for 50 ms
    if (reading != buttonState) {                                 // and it differs from the
    accepted state
      buttonState = reading;
      if (buttonState == LOW) pressed = true;                     // falling edge = one press
    }
  }
  lastReading = reading;
  return pressed;
}
```

Three variables, three ideas: the raw reading restarts a stopwatch; the stopwatch must reach 50 ms; only then is the new level accepted, and the press is reported on the HIGH→LOW *edge*, once. Call it every `loop()` even when you do not need the result, so it keeps tracking. For several buttons, put the three variables in a `struct Button` and pass it by reference. The lecture suggests keeping the function in a `tools.h` file.

Key insight. Level versus edge. "LED on while held" needs the *level* (ex09). "Count presses", "toggle", "next state" need the *edge*: the transition from released to pressed, reported once (ex10, ex11). Every FSM in part G consumes edges.

5 E. Time: `delay()` versus `millis()`

`delay(ms)` stops the program for `ms` milliseconds: nothing else runs, no button is read, no serial byte is processed (the 64-byte receive buffer still fills in the background, then overflows). `millis()` returns the

number of milliseconds since reset as an `unsigned long`; `micros()` the microseconds (4 μs resolution). Neither waits.

Method: The non-blocking pattern

```
const unsigned long INTERVAL = 1000;
unsigned long previous = 0;

void loop() {
  unsigned long now = millis();           // read once per loop
  if (now - previous >= INTERVAL) {       // "has INTERVAL elapsed since the last time?"
    previous = now;                       // (or previous += INTERVAL to avoid drift)
    // the periodic action
  }
  // everything else runs thousands of times per second meanwhile
}
```

One `previous` per periodic task. Several tasks = several independent timers in the same `loop()` (ex06, ex08). A **one-shot**: record `startTime` on the trigger, set a flag, and in `loop()` test `running && now - startTime >= DURATION` (ex12).

Warning / Gotcha: Why `now - previous >= INTERVAL` and never `now >= previous + INTERVAL`

`millis()` wraps to 0 after 2^{32} ms = 49.7 days. Unsigned subtraction is computed modulo 2^{32}, so `now - previous` is still the true elapsed time across the wrap: $5 - 4\,294\,967\,290 \equiv 11 \pmod{2^{32}}$. The addition form compares against a value that has wrapped and the timer stops for 49 days. This also requires `unsigned long` for every timestamp: a signed or 16-bit variable breaks the arithmetic.

Drift. `previous = now` schedules the next event from *when the loop noticed*, so a slow loop iteration shifts all later events. `previous += INTERVAL` keeps the schedule anchored to the start (used in ex07). Both are fine for LEDs; the second matters for clocks.

6 F. Serial Communication (UART)

Concept: UART basics

The ATmega328P has one hardware UART on pins 0 (RX) and 1 (TX); on the Uno a USB bridge chip connects it to the PC, where it appears as a serial port read by the Serial Monitor. A byte is sent as 10 bits (start, 8 data, stop), least significant bit first, at the **baud rate**: at 9600 bit/s one byte takes 1.04 ms, so printing a 20-character line blocks about 21 ms once the 64-byte transmit buffer is full. Both ends must use the same baud rate. Once `Serial.begin()` is called, pins 0 and 1 are not available as digital I/O.

Interface / API: The API

`Serial.begin(9600)`	start the UART; `Serial.end()` frees pins 0/1
`Serial.print(x)` / `println(x)`	send the human-readable **text** of x: `print(65)` sends the characters `'6'`,`'5'`; `print(3.14159, 2)` sends `3.14`; `print(x, HEX)`; `println` adds `"\r\n"`
`Serial.write(b)`	send the **raw byte** b: `write(65)` sends one byte 65, shown as `A` by the monitor
`Serial.available()`	number of bytes waiting in the 64-byte receive buffer (0 = nothing)
`Serial.read()`	remove and return one byte as an `int`, or −1 if empty
`Serial.peek()`	return the next byte *without* removing it (−1 if empty)
`Serial.parseInt()`	skip non-digits, read the digits of the next integer, return it as a `long` (0 after a 1 s timeout)
`Serial.parseFloat()`	same for a decimal number
`Serial.readStringUntil('\n')`	collect characters into a `String` until the terminator (not included) or a 1 s timeout
`Serial.readBytes(buf, n)`	fill a buffer with n bytes

Worked example: `print` versus `write`, the lab's table

Value	`print(value)` shows	bytes on the wire	`write(value)` shows	byte on the wire
48	`48`	52 56	`0`	48
49	`49`	52 57	`1`	49
50	`50`	53 48	`2`	50
65	`65`	54 53	`A`	65
66	`66`	54 54	`B`	66

`print` converts the number to its decimal digits and sends their ASCII codes; `write` sends the number itself as one byte. To ship a small number from Arduino A to Arduino B and get it back with `Serial.read()`, use `write`: `read()` returns exactly the byte that was written. With `print(10)`, B would read 49 then 48. (ASCII: `'0'`..`'9'` are 48..57, `'A'` is 65, `'a'` is 97, newline is 10.)

Worked example: `read`, `parseInt`, `peek` (Serial lab, exercise 4)

Type `5`: `read()` returns 53, the code of the character `'5'`; `53 - '0'` is the digit. Type `25`: one `read()` returns 50 (`'2'`); the `'5'` is still in the buffer: `read()` never assembles a number. `parseInt()` returns 25, and 125 for `125`: the tool for multi-digit numbers. Type `A`: `peek()` returns 65 twice and `available()` stays 1; after `read()` (65) `available()` is 0.

Function	Role	Removes the byte?	Suitable for 125?
`read()`	next byte as a code	yes	no (gives `'1'`)
`parseInt()`	next integer written in decimal	yes (the digits)	yes
`peek()`	look at the next byte	no	no

A single command like `A`: `read()`. A number like 125: `parseInt()`. Check without consuming: `peek()`. Read and remove one character: `read()`.

Method: Receiving without blocking: three patterns

```
if (Serial.available() > 0) { char c = Serial.read(); /* one-character commands: '1', '0', 'A' */ }

if (Serial.available() > 0 && isDigit(Serial.peek())) { long n = Serial.parseInt(); }   // numbers;
     discard the '\n' separately

if (Serial.available() > 0) {                          // line protocol: "R128", "ON", "SET 20"
  String line = Serial.readStringUntil('\n');          // blocks at most until the '\n' arrives (1 s
    timeout)
  line.trim();                                         // strip '\r' and spaces
  if (line == "ON") ... else if (line.startsWith("R")) { int v = line.substring(1).toInt(); }
}
```

Debug prints: wrap them in macros so they vanish in the release build: `#define DEBUG ... #ifdef DEBUG #define DPRINTLN(x) Serial.println(x) #else #define DPRINTLN(x) #endif`. And never print on every `loop()` iteration ("floods the monitor"): print on changes or on a `millis()` timer.

6.1 Arduino to Arduino

Wire A.TX(1) → B.RX(0), B.TX(1) → A.RX(0), and **GND to GND** (no common ground, no signal). Same baud on both. Each board talks to the other exactly as it talks to the PC: there is only one UART, so while the boards are linked the Serial Monitor cannot be used on those pins. Protocol patterns from the Serial lab, all in the kit:

- **one-way stream** (ex 5–8): A `write`s a counter every 2–3 s; B `read`s and acts (LED when ≥ 10) — ex18.
- **request / acknowledgement** (ex 9–11): A sends a value; B answers 1 or 0 with `write`; A reads the answer and lights its LED — ex19 and `labs/03_serial/ex11_A.ino`, `ex11_B.ino`.

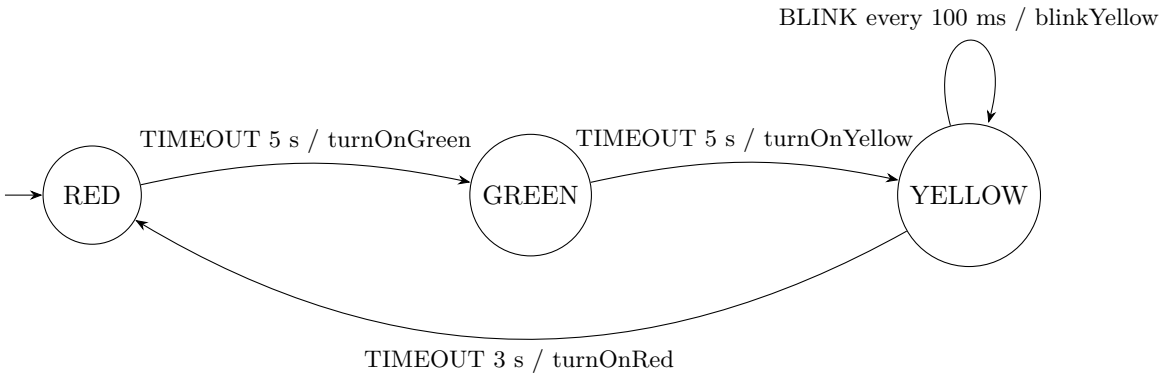
Keep the receive side non-blocking (`available()` then `read()`) so a board that is also blinking or counting never stalls.

7 G. Finite State Machines

As soon as a system has several modes and several things that can happen, nested `if`s on flags collapse into code nobody can read. The embedded answer is to make the *mode* explicit.

Concept: FSM = (S, E, δ, s_0)

A finite state machine has a finite set of **states** S (the situation the system is in: RED, IDLE, ALERT), a finite set of **events** E (things that happen: TIMEOUT, BUTTON_PRESSED, MOTION), a **transition function** $\delta : S \times E \to S$ (in state s, event e leads to state $\delta(s, e)$), and an initial state s_0. A transition may carry an **action** (turn an LED on, start a timer). A transition from a state to itself is an *internal* or *self* transition: the state does not change but an action runs (YELLOW $\xrightarrow{\text{BLINK}}$ YELLOW toggles the LED).



Label each arrow "event / action". Read the diagram as a table: it *is* the transition table.

From	Event	To	Action
RED	TIMEOUT (after 5 s)	GREEN	turnOnGreen()
GREEN	TIMEOUT (after 5 s)	YELLOW	turnOnYellow()
YELLOW	BLINK (every 100 ms for 3 s)	YELLOW	blinkYellow()
YELLOW	TIMEOUT (after 3 s)	RED	turnOnRed()

7.1 Three ways to write the same machine

The lecture's LED example: IDLE (off) → ACTIVE (on) → BLINKING (blinks, returns to IDLE on a press or after 2 s).

Method: 1. State-based reasoning (recommended for hand-written code)

"Where am I?" first, "what does the event mean here?" second. One `case` per state; inside it, the events.

```
enum class State { IDLE, ACTIVE, BLINKING };
State currentState = State::IDLE;
unsigned long stateEnteredAt = 0;

void loop() {
  bool pressed = buttonPressed(PB);            // the EVENT, debounced, true once per press
  unsigned long now = millis();
  switch (currentState) {
    case State::IDLE:
      if (pressed) { turnOn(); currentState = State::ACTIVE; }
      break;
    case State::ACTIVE:
      if (pressed) { currentState = State::BLINKING; stateEnteredAt = now; lastBlink = now; }
      break;
    case State::BLINKING:
      if (pressed || now - stateEnteredAt >= TIMEOUT) {   // two events, same transition
        turnOff(); currentState = State::IDLE;
      } else blink();                                      // "stay" action
      break;
  }
}
```

Every `case` answers the same three questions: which events move me out (and what to do on the way), and what to keep doing while I stay.

2. Event-based reasoning swaps the nesting: `switch (pressed)` outside, `switch (currentState)` inside. Same behaviour, but the code for one state is scattered across the event branches, which is why the course moves to the state-based form. The lecture's version of both also uses `delay(200)` as a crude debounce and `delay(100)` inside the blink; the kit's ex21 is the non-blocking rewrite.

Method: 3. Table-driven FSM (the traffic-light lab)

The behaviour becomes *data*; the engine is five lines and never changes.

```
enum class State { GREEN, YELLOW, RED };
enum class Event { TIMEOUT, BLINK };
struct Transition { State from; Event event; State to; void (*action)(); };   // the 4-tuple, with
    a function pointer

Transition transitions[] = {                        // one row per arrow of the diagram
  { State::RED,    Event::TIMEOUT, State::GREEN,  &turnOnGreen  },
  { State::GREEN,  Event::TIMEOUT, State::YELLOW, &turnOnYellow },
  { State::YELLOW, Event::BLINK,   State::YELLOW, &blinkYellow  },   // self-loop
  { State::YELLOW, Event::TIMEOUT, State::RED,    &turnOnRed    },
};
const int NB = sizeof(transitions) / sizeof(transitions[0]);

void processEvent(Event event = Event::TIMEOUT) { // default argument: see the prototype trap in
    part B
  for (int i = 0; i < NB; i++) {
    if (transitions[i].from == currentState && transitions[i].event == event) {
      transitions[i].action();                      // run the action
      currentState = transitions[i].to;             // move
      break;                                        // first matching row wins; without it, later
    rows could fire too
    }
  }                                                 // no row: the event is ignored in this state
}
```

`loop()` only *generates events* from sensors and the clock and feeds them to `processEvent()`. Adding a pedestrian button is two new rows and one new `if` in `loop()`; the engine is untouched (ex23, and the NIGHT state in `labs/06_traffic_fsm/task5_night.ino`).

7.2 Timed states without `delay()`

The lab's Listing 1 holds RED and GREEN with `delay(5000)` inside the actions. It works, but for 10 of every 13 seconds the CPU is frozen: no button, no serial, no second task. The fix is the one-shot timer of part E applied to states:

```
unsigned long stateEnteredAt = 0;                          // set by every entry action:
    stateEnteredAt = millis();
unsigned long durationOf(State s) { ... RED 5000, GREEN 5000, YELLOW 3000 ... }

void loop() {
  unsigned long now = millis();
  if (now - stateEnteredAt >= durationOf(currentState)) processEvent(Event::TIMEOUT);   // the clock
    generates TIMEOUT
  else if (currentState == State::YELLOW)               processEvent(Event::BLINK);     // stay action
}
```

Full code: ex22. Measured in the simulator: red 5.000 s, green 5.000 s, yellow 3.000 s, cycle 13.000 s, and the loop runs about 10 000 times per second between transitions, free for other work.

Method: Designing an FSM: the recipe

1. List the **states**: the distinguishable situations, each with one responsibility and a clear set of outputs (which LEDs are on, is the buzzer sounding). Name them with an `enum class`.
2. List the **events**: button edges, sensor levels, serial commands, and one `TIMEOUT` per timed state.
3. Draw the **diagram**; write the **table** (from, event, to, action). Check every state handles every event it should, and decide explicitly which events are ignored.
4. Write one **entry action** per state that sets *all* outputs (every LED, not only the one that changes) and records `stateEnteredAt`. Entry actions make the state's outputs independent of the path that led there.
5. Write **stay actions** for things that repeat inside a state (blinking, beeping, counting).
6. In `loop()`: read inputs (debounced edges), generate events, dispatch (switch or table). Never `delay()`.
7. Instrument: print each transition once (`"RED + TIMEOUT -> GREEN"`), not the state every loop.

Warning / Gotcha: FSM pitfalls seen in the course material

- **Blocking actions**: `delay(5000)` inside `turnOnRed()` means a pedestrian press during RED is lost (Listing 1, Task 4).
- **No entry action for the initial state**: Listing 1 starts in RED but never calls `turnOnRed()`, so the first state actually displayed is GREEN (verified in the simulator: green from 0 to 5 s, then yellow, then red). Call the entry action in `setup()`.
- **Missing `break`** in `processEvent`: after RED→GREEN the loop continues scanning, finds GREEN+TIMEOUT, and jumps to YELLOW in the same call: two transitions on one event.
- **Flooding**: `Serial.println("IDLE state")` on every loop iteration sends 1000 lines a second and the serial buffer blocks the loop.
- **Declaring a variable under a `case`** without braces (the Canvas "FSM State Based Reasoning" code): compile error.
- **Reading the button with `delay(200)`** as a debounce makes every state react 200 ms late and misses short presses.
- **A level used as an event**: `if (digitalRead(PB) == LOW)` in a state fires on every loop while the button is held, cycling through all states in a few ms. Use the debounced edge.

8 H. The Labs, Solved

Code for every task is in `kit/labs/` (see its `README.md` for the task-to-file map) or in `kit/solutions/`; everything below was run in the simulator. This part gives the measurements, tables and written answers.

8.1 Lab: the on-board LED (pin 13)

Task 1. With Listing 1: $t_{ON} = 1000$ ms, $t_{OFF} = 1000$ ms, $T = 2000$ ms, $f = 0.5$ Hz. **Task 2.** Both delays 250 ms: $T = 500$ ms, $f = 2$ Hz (four times faster). 100 ms ON / 900 ms OFF: $T = 1$ s, $D = 100/1000 = 10\%$. **Task 3** SOS: `solutions/ex02_sos.ino`. **Task 4** bonus (2 Hz + blink counter every 5 s): `labs/01_internal_led/task4_bonus.ino`, prints `Blinks so far:  10, 20, ...`

Answers: Post-lab questions

1. `LED_BUILTIN` is a constant defined by the Arduino core as 13 on the Uno; header pin D13 is routed on the PCB to the microcontroller's port pin PB5 (bit 5 of port B); the "L" LED hangs on that same trace through a 1 kΩ resistor. Writing `PORTB` bit 5, `digitalWrite(13)` and `digitalWrite(LED_BUILTIN)` are the same action.
2. $I = (5 - 2)/330 = 9.1$ mA: still far below the 40 mA maximum (and below the 20 mA recommendation): safe.
3. Without the second `delay()`, the LED is turned LOW and, a few microseconds later, HIGH again at the next `loop()`: it is off for about 5 μs per 1 s cycle. The eye sees it permanently on.
4. D13 is also SPI's SCK clock line. When an SPI device is used, the line toggles at MHz rates: the LED flickers meaninglessly and its resistor/capacitance load the clock edge; in older boards the LED buffer could even pull the line and corrupt the clock. Use another pin as an activity indicator when SPI is in use.
5. The `millis()` version never freezes the CPU: between toggles it can read buttons, sensors and the serial port, and several such tasks can share one `loop()`. The `delay()` version can do exactly one thing.

8.2 Lab: blinking an external LED

Task 1. Listing 1: $t_{ON} = t_{OFF} = 500$ ms, $T = 1$ s, $f = 1$ Hz. With 100 ms delays, $f = 5$ Hz: still visible flicker (persistence of vision starts near 50 Hz). **Task 2** (opposition): `solutions/ex04_opposition.ino`. **Task 3** (fading): `solutions/ex05_fade.ino`.

Answers: Post-lab questions

1. Shorter leg; flat edge on the rim of the plastic body; the larger internal plate (the "anvil") seen through the lens is the cathode. In the symbol, the bar the triangle points to.
2. $R = (5 - 2)/0.020 = 150\ \Omega$, which is an E12 value (100, 120, **150**, 180, 220, ...). 180 Ω would give 16.7 mA if one prefers margin.
3. Without the resistor, the pin driver and the LED form a near short circuit: the current is limited only by the driver (tens of mA beyond the 40 mA rating). Tinkercad shows the LED exploding ("magic smoke"); on hardware the LED or the pin driver is damaged.
4. Nothing lights: the sketch still drives pin 5, and pin 4 is left as a high-impedance input. Change `LED_PIN` to 4 or move the wire back.
5. Pin 5 is connected to a timer compare output (Timer 0, OC0B); `analogWrite` programs that timer to produce a 980 Hz square wave on it. Pin 4 has no timer output, so `analogWrite(4, x)` falls back to a plain HIGH/LOW. During fading, the pin carries a square wave of fixed frequency whose HIGH fraction grows from 0% to 100% and back: a pulse train of constant period and growing width.

8.3 Lab: Serial communication

Ex 1: `solutions/ex13`. Ex 2: `ex14`. Ex 3: table in part F; code `labs/03_serial/ex3_print_vs_write.ino`. Ex 4: analysis and table in part F; code `ex4_read_parseint_peek.ino`. Ex 5–6: `ex5_6_A/B.ino`; Ex 7: `ex7_A/B.ino`; Ex 8: `solutions/ex18`; Ex 9–10: `solutions/ex19`; Ex 11: `ex11_A/B.ino` (A's LED lights at 12 s, when the count 10 is received and acknowledged).

Key insight. Ex 3 question 11: to send a small number from A to B and read it with `Serial.read()`, use `Serial.write()`. `print()` would send digits.

8.4 Lab: the floating input and pull resistors

Observation table: released, expected a stable level, observed random "Pressed"/"Released" flicker that changes when a hand approaches the wires; pressed, pin forced to 0 V, observed a steady reading, but Listing 1 prints **Released** because its logic is inverted (pressed = LOW enters the `else`). Solution A (`INPUT_PULLUP`, pressed = LOW): `labs/04_floating_pin/solutionA_internal_pullup.ino`. Solution B (external 10 kΩ pull-down, button to 5 V, pressed = HIGH): `solutionB_external_pulldown.ino`. Bonus debounce: `solutions/ex10`.

Answers: Post-lab questions

1. No component, no extra wire, nothing to get wrong on the breadboard, and the button simply goes to GND; one line of code enables it. An external pull-down costs a resistor and two more connections.
2. Capacitive coupling: your body carries 50 Hz mains hum and static charge; through the pin's ≈ 100 MΩ input impedance a few picoamps are enough to lift the node above the 2.5 V threshold.
3. `delay()` freezes the CPU for the debounce time on every press: inputs, sensors and communication are ignored during it, timing of other tasks drifts, and a system with several buttons becomes unresponsive. The `millis()` debounce consumes no time at all.

8.5 Lab: `millis()` and the bouncing problem

Ex 1: `labs/05_millis/ex1_elapsed.ino` (ms, s, m:ss every 2 s) and `solutions/ex07`. Ex 2: `ex2_led12.ino`. Ex 3: `solutions/ex06` (two timestamps because each LED has its own schedule; one shared timestamp would be reset by whichever LED toggles first, breaking the other's period). Ex 4: `solutions/ex08`. Ex 5: `solutions/ex12`. Ex 6: `ex6_debounce_counter_led.ino` (bouncing produces several falling edges per press; the 50 ms stability rule keeps one). Ex 7: `ex7_three_leds.ino` (timers in arrays; the button toggles blinking, the "System active" message keeps its own timer). Ex 8 mini-project: `solutions/ex25`.

Answers: Review questions

1. `delay(n)` stops the program for n ms; `millis()` returns the time since reset and returns immediately. One wastes time, the other measures it.
2. `millis()` returns a 32-bit unsigned value (up to 4 294 967 295); an `int` (16-bit, max 32 767) overflows after 32 s and a signed type breaks the wrap-around subtraction.
3. `micros()` returns microseconds (4 μs resolution, wraps every 71.6 min); `millis()` milliseconds (wraps every 49.7 days).
4. Subtraction in unsigned arithmetic gives the elapsed time even after the counter wraps; `>=` rather than `==` because the loop rarely lands on the exact millisecond.
5. Yes: one `previous` variable per timer, all compared against the same `millis()`.
6. Because no task ever blocks the others: every input is polled thousands of times a second and every output keeps its own schedule.
7. After 2^{32} ms ≈ 49.7 days it restarts at 0. Code written as `now - previous >= interval` with `unsigned long` keeps working; code written as `now >= previous + interval` or with signed variables stalls.

8.6 Lab: traffic lights with an FSM

Task 1 (Listing 1 as given, measured in the simulator): the first state shown is GREEN, not RED, because the initial RED has no entry action; then yellow 3 s, red 5 s, green 5 s, ...: RED 5.000 s, GREEN 5.000 s, YELLOW 3.000 s blinking, cycle 13.000 s. **Task 2**: `labs/06_traffic_fsm/task2_logged.ino` logs `RED + TIMEOUT -> GREEN`, `GREEN + TIMEOUT -> YELLOW`, `YELLOW + TIMEOUT -> RED`. **Task 3**: Green 10 s, Yellow 2 s, Red 8 s means editing the two `delay(5000)` (10000 and 8000) and the `3000` in `loop()` (2000); in the non-blocking version, three constants. New period $10 + 2 + 8 = 20$ s. **Task 4**: `solutions/ex22`. **Task 5** NIGHT: `task5_night.ino` (4 states, 9 transitions: each day state gets a NIGHT_FALL arrow to NIGHT, NIGHT has a BLINK self-loop and DAY_BREAK back to RED). Pedestrian button: `solutions/ex23`.

Answers: Post-lab questions

1. Three states, four transitions: RED→GREEN, GREEN→YELLOW, YELLOW→YELLOW (BLINK), YELLOW→RED (the diagram in part G).
2. Its source and target are the same state: the machine stays in YELLOW; only the action (toggling the LED) runs.
3. After the first matching row is processed, the loop keeps scanning with the *new* `currentState`; if a later row matches it (GREEN + TIMEOUT after RED + TIMEOUT), a second transition fires in the same call: RED would jump to YELLOW in one step.
4. $T = 200$ ms, $f = 5$ Hz.
5. The switch/table version separates *what can happen* (the table) from *how events are produced* (the loop) and from *what each state does* (the actions). Adding a state or event is adding rows; the code matches the diagram line by line, so it can be checked by reading; nested `if`/`delay` code hides the states in flags and cannot react during delays.
6. Each entry action records `stateEnteredAt = millis();` `loop()` computes `millis() - stateEnteredAt >= durationOf(currentState)` and only then calls `processEvent(TIMEOUT);` while in YELLOW it calls `processEvent(BLINK)` instead. The refactored `loop()` is in ex22.

8.7 Lecture-2 labs and Canvas assignments

- **Simple Traffic Light V2 / with blinking Yellow V2**: `labs/07_slide_labs/assignment_simple_traffic_light.ino` (red 5 s, green 5 s, yellow 2 s, delay-based) and `assignment_traffic_blinking_yellow.ino` (yellow 100/100 ms for 3 s).
- **Lab 2 Smart-home lighting** (OFF → ON → BLINK 500 ms → OFF with one button): `solutions/ex20`.
- **Lab 3 Three buttons, three pull configurations**: `three_buttons.ino`; pin 2 external pull-down (pressed HIGH), pin 3 external pull-up (pressed LOW), pin 4 internal pull-up (pressed LOW).
- **Lab 4 Store alarm (PIR + buzzer)**: `solutions/ex24`. The slide wires the PIR output and the red LED both to pin 12; the solution uses PIR 2, red 12, green 13, buzzer 11.
- **FSM lab, solutions 1 and 2**: the lecture's code (blocking); the non-blocking equivalent with transition logging is `solutions/ex21`.
- **Canvas assignment "FSM State Based Reasoning"**: the given code has eight defects: `TIMEOUT` undefined; `debouncingButton()` never written; `turnON` defined but `turnOn` called; the `{` after `switch(currentState)` missing (and its closing brace); `intialTime` typo; `unsigned long now` declared under a `case` without braces; `blink()` called at the ACTIVE→BLINK transition instead of initialising `lastBlink`; and the LED left on when entering BLINK. Fixed version with each fix marked: `labs/07_slide_labs/fsm_state_based_fixed.ino`.
- **Advanced FSM (transition table with PRESSED / KEEPBLINKING / TIMEOUT)**:

advanced_fsm_table.ino.

- **Piano lab** (three keys, `tone()`): solutions/ex27.

9 I. The 27 Exercises and Their Solutions

Statements are in kit/exercises/exNN_*/README.md; the simulator grades each one (`python3 check.py exNN`). The listings below are the reference solutions, exactly the files in `kit/solutions/`, all of which pass the checker and compile with `avr-g++`.

#	Statement (short)	Checked
01	Blink pin 13, 500/500 ms	period, duty, pinMode
02	SOS: dots 200 ms, dashes 600 ms, gaps 200/600, pause 1 s	sequence of ON durations
03	Pin 5: 100 ms ON, 900 ms OFF	durations
04	Pins 5 and 13 in opposition, 500 ms	levels always differ
05	PWM fade 0..255 step 5, 30 ms/step	analogWrite sequence and timing
06	Pin 12 every 500 ms, pin 11 every 1500 ms, no delay	both periods, no delay()
07	Print elapsed time as m:ss every 2 s	lines and times
08	Product every 3 s, LED 500 ms, stop at 20	20 lines, LED continues
09	LED 12 follows button 3 (INPUT_PULLUP)	levels during presses
10	Count presses, 50 ms debounce, bouncing input	exactly one count per press
11	Each press toggles LED 12	toggles, hold counts once
12	Press starts a 5 s run, presses ignored meanwhile	ON windows, messages
13	Send 5, 6, 7, ... every 3 s	lines and times
14	'1'/'0' from the PC drive LED 13	levels after each command
15	Echo byte as print then write	text and raw bytes
16	parseInt of 25, 125, 7	"got N" lines
17	Line commands ON / OFF / BLINK	LED, blinking, unknown line
18	Two boards: counter with write(), LED when ≥ 10	bytes, B's LED
19	Two boards: request 15, ack 1/0	bytes both ways, A's LED
20	FSM OFF/ON/BLINK by button	LED per state
21	FSM IDLE/ACTIVE/BLINKING with 2 s timeout	timing, transition lines
22	Table-driven traffic light, non-blocking	phase durations, no delay()
23	Pedestrian request on the traffic light	shortened green, early green, ignored in yellow
24	PIR alarm STANDBY/ALERT/COOLDOWN	beeps, cooldown, lines
25	Industrial machine mini-project	products, LEDs, restart
26	Blink with DDRB/PORTB only	register-level pinMode, period
27	Three-key piano with tone()	frequencies per key

ex01 Blink

```
// ex01 -- Blink at 1 Hz (period 1000 ms, duty cycle 50 %)
const int LED = LED_BUILTIN;   // pin 13 = PB5 on the ATmega328P

void setup() {
  pinMode(LED, OUTPUT);        // push-pull output driver
}

void loop() {
  digitalWrite(LED, HIGH);     // 5 V on the pin: current flows through the 1 kohm resistor and the LED
  delay(500);                  // blocking wait: the CPU does nothing else for 500 ms
```

```
  digitalWrite(LED, LOW);
  delay(500);
}
```

ex02 SOS

```
// ex02 -- SOS in Morse code with helper functions
const int LED = LED_BUILTIN;
const int DOT = 200, DASH = 600, GAP = 200, LETTER_GAP = 600, WORD_PAUSE = 1000;

void flash(int ms) {              // one symbol: ON for ms, then the intra-letter gap
  digitalWrite(LED, HIGH);
  delay(ms);
  digitalWrite(LED, LOW);
  delay(GAP);
}
void dot()  { flash(DOT); }
void dash() { flash(DASH); }
void letterGap() { delay(LETTER_GAP - GAP); }   // flash() already waited GAP

void setup() { pinMode(LED, OUTPUT); }

void loop() {
  dot(); dot(); dot();  letterGap();     // S
  dash(); dash(); dash(); letterGap();   // O
  dot(); dot(); dot();                   // S
  delay(WORD_PAUSE - GAP);               // 1 s pause after the word
}
```

ex03 Asymmetric blink

```
// ex03 -- 100 ms ON / 900 ms OFF: duty cycle 10 %
const int LED_PIN = 5;

void setup() { pinMode(LED_PIN, OUTPUT); }

void loop() {
  digitalWrite(LED_PIN, HIGH);
  delay(100);
  digitalWrite(LED_PIN, LOW);
  delay(900);
}
```

ex04 Opposition

```
// ex04 -- opposition: write the two pins with opposite values in the same step
const int EXT_LED = 5;
const int INT_LED = LED_BUILTIN;

void setup() {
  pinMode(EXT_LED, OUTPUT);
  pinMode(INT_LED, OUTPUT);
}

void loop() {
  digitalWrite(EXT_LED, HIGH);
  digitalWrite(INT_LED, LOW);
  delay(500);
  digitalWrite(EXT_LED, LOW);
  digitalWrite(INT_LED, HIGH);
  delay(500);
}
```

ex05 PWM fade

```ino
// ex05 -- PWM fading. analogWrite(pin, d) outputs a ~490 Hz square wave with duty d/255;
// the eye averages it into a brightness level.
const int LED_PIN = 5;   // must be a PWM pin: 3, 5, 6, 9, 10, 11

void setup() { pinMode(LED_PIN, OUTPUT); }

void loop() {
  for (int duty = 0; duty <= 255; duty += 5) {    // fade in
    analogWrite(LED_PIN, duty);
    delay(30);
  }
  for (int duty = 250; duty >= 0; duty -= 5) {    // fade out (255 was already shown)
    analogWrite(LED_PIN, duty);
    delay(30);
  }
}
```

ex06 Two timers

```ino
// ex06 -- two independent non-blocking timers: one 'previous' timestamp per task
const int GREEN = 12;
const int RED = 11;
const unsigned long GREEN_INTERVAL = 500;
const unsigned long RED_INTERVAL = 1500;

unsigned long prevGreen = 0, prevRed = 0;   // unsigned long: millis() is 32-bit
bool greenOn = false, redOn = false;

void setup() {
  pinMode(GREEN, OUTPUT);
  pinMode(RED, OUTPUT);
}

void loop() {
  unsigned long now = millis();           // read the clock once per loop
  if (now - prevGreen >= GREEN_INTERVAL) {
    prevGreen = now;
    greenOn = !greenOn;
    digitalWrite(GREEN, greenOn);
  }
  if (now - prevRed >= RED_INTERVAL) {    // independent: its own timestamp
    prevRed = now;
    redOn = !redOn;
    digitalWrite(RED, redOn);
  }
}
```

ex07 Elapsed time

```ino
// ex07 -- periodic printing with millis(); integer arithmetic to split seconds into m:ss
const unsigned long INTERVAL = 2000;
unsigned long previous = 0;

void setup() {
  Serial.begin(9600);
}

void loop() {
  unsigned long now = millis();
  if (now - previous >= INTERVAL) {
    previous += INTERVAL;                 // += keeps the schedule exact (no drift from loop latency)
    unsigned long totalSeconds = now / 1000;
```

```
    unsigned long minutes = totalSeconds / 60;
    unsigned long seconds = totalSeconds % 60;
    Serial.print(minutes);
    Serial.print(':');
    if (seconds < 10) Serial.print('0');  // zero padding
    Serial.println(seconds);
  }
}
```

ex08 Production counter

```
// ex08 -- two tasks in one loop(): a counter task and an LED task, each with its own timer
const int LED = 12;
const unsigned long PRODUCT_INTERVAL = 3000;
const unsigned long LED_INTERVAL = 500;
const int MAX_PRODUCTS = 20;

unsigned long prevProduct = 0, prevLed = 0;
int counter = 0;
bool ledOn = false;

void setup() {
  pinMode(LED, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  unsigned long now = millis();

  // task 1: production counter, stops at 20 but never blocks the loop
  if (counter < MAX_PRODUCTS && now - prevProduct >= PRODUCT_INTERVAL) {
    prevProduct = now;
    counter++;
    Serial.print("Product produced : ");
    Serial.println(counter);
  }

  // task 2: LED, fully independent
  if (now - prevLed >= LED_INTERVAL) {
    prevLed = now;
    ledOn = !ledOn;
    digitalWrite(LED, ledOn);
  }
}
```

ex09 Button and LED

```
// ex09 -- INPUT_PULLUP: released = HIGH, pressed = LOW. Invert the reading to get "pressed".
const int BUTTON = 3;
const int LED = 12;

void setup() {
  pinMode(BUTTON, INPUT_PULLUP);   // enables the ~20-50 kohm internal resistor to 5 V
  pinMode(LED, OUTPUT);
}

void loop() {
  bool pressed = (digitalRead(BUTTON) == LOW);
  digitalWrite(LED, pressed ? HIGH : LOW);
}
```

ex10 Debounced counter

```
// ex10 -- the lecture's debounce algorithm: a change is accepted only after it has been stable for
    DEBOUNCE_DELAY
const int BUTTON = 3;
const unsigned long DEBOUNCE_DELAY = 50;

int buttonState = HIGH;        // last *stable* state (HIGH = released)
int lastReading = HIGH;        // last *raw* reading
unsigned long lastDebounceTime = 0;
int count = 0;

// Returns true exactly once per valid press (falling edge HIGH -> LOW)
bool buttonPressed() {
  int reading = digitalRead(BUTTON);
  bool pressed = false;
  if (reading != lastReading) {          // any change (real or bounce) restarts the timer
    lastDebounceTime = millis();
  }
  if (millis() - lastDebounceTime > DEBOUNCE_DELAY) {   // stable long enough
    if (reading != buttonState) {        // and different from the accepted state
      buttonState = reading;
      if (buttonState == LOW) pressed = true;
    }
  }
  lastReading = reading;
  return pressed;
}

void setup() {
  pinMode(BUTTON, INPUT_PULLUP);
  Serial.begin(9600);
}

void loop() {
  if (buttonPressed()) {
    count++;
    Serial.print("count=");
    Serial.println(count);
  }
}
```

ex11 Toggle

```
// ex11 -- edge detection: act on the press event, not on the level
const int BUTTON = 3;
const int LED = 12;
const unsigned long DEBOUNCE_DELAY = 50;

int buttonState = HIGH, lastReading = HIGH;
unsigned long lastDebounceTime = 0;
bool ledOn = false;

bool buttonPressed() {
  int reading = digitalRead(BUTTON);
  bool pressed = false;
  if (reading != lastReading) lastDebounceTime = millis();
  if (millis() - lastDebounceTime > DEBOUNCE_DELAY && reading != buttonState) {
    buttonState = reading;
    if (buttonState == LOW) pressed = true;
  }
  lastReading = reading;
  return pressed;
}

void setup() {
  pinMode(BUTTON, INPUT_PULLUP);
  pinMode(LED, OUTPUT);
```

```
}

void loop() {
  if (buttonPressed()) {          // true only on the falling edge, once per press
    ledOn = !ledOn;
    digitalWrite(LED, ledOn);
  }
}
```

ex12 Push-button timer

```
// ex12 -- one-shot timer: remember when it started, stop when RUN_TIME has elapsed
const int BUTTON = 3;
const int LED = 12;
const unsigned long RUN_TIME = 5000;
const unsigned long DEBOUNCE_DELAY = 50;

int buttonState = HIGH, lastReading = HIGH;
unsigned long lastDebounceTime = 0;
bool running = false;
unsigned long startTime = 0;

bool buttonPressed() {
  int reading = digitalRead(BUTTON);
  bool pressed = false;
  if (reading != lastReading) lastDebounceTime = millis();
  if (millis() - lastDebounceTime > DEBOUNCE_DELAY && reading != buttonState) {
    buttonState = reading;
    if (buttonState == LOW) pressed = true;
  }
  lastReading = reading;
  return pressed;
}

void setup() {
  pinMode(BUTTON, INPUT_PULLUP);
  pinMode(LED, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  bool pressed = buttonPressed();          // always call it, so its state keeps tracking the button
  unsigned long now = millis();
  if (!running) {
    if (pressed) {                         // a press is only honoured while stopped
      running = true;
      startTime = now;
      digitalWrite(LED, HIGH);
      Serial.println("Machine started");
    }
  } else if (now - startTime >= RUN_TIME) {
    running = false;
    digitalWrite(LED, LOW);
    Serial.println("Machine stopped");
  }
}
```

ex13 Serial counter

```
// ex13 -- Arduino -> PC. println() sends the decimal digits as ASCII text plus "\r\n".
int nb_pieces = 5;

void setup() {
  Serial.begin(9600);        // 9600 bits/s: about 1 ms per character
}
```

```
7
8  void loop() {
9    Serial.println(nb_pieces);
10   nb_pieces++;
11   delay(3000);
12 }
```

ex14 Serial command

```
1  // ex14 -- PC -> Arduino. The received byte is a CHARACTER: compare with '1', not with 1.
2  const int LED = LED_BUILTIN;
3
4  void setup() {
5    Serial.begin(9600);
6    pinMode(LED, OUTPUT);
7  }
8
9  void loop() {
10   if (Serial.available() > 0) {        // never block waiting for data
11     char cmd = Serial.read();          // one byte, removed from the 64-byte buffer
12     if (cmd == '1') {
13       digitalWrite(LED, HIGH);
14     } else if (cmd == '0') {
15       digitalWrite(LED, LOW);
16     }
17     // '\n', '\r' and anything else: ignored
18   }
19 }
```

ex15 print versus write

```
1  // ex15 -- print(v): the decimal TEXT of v ("65" = bytes 54 53).  write(v): the single raw byte v (65 = 'A').
2  void setup() {
3    Serial.begin(9600);
4  }
5
6  void loop() {
7    if (Serial.available() > 0) {
8      int b = Serial.read();      // 0..255 (int, because -1 means "nothing")
9      Serial.print(b);            // "65"
10     Serial.print(' ');
11     Serial.write(b);            // 'A'
12     Serial.println();
13   }
14 }
```

ex16 parseInt

```
1  // ex16 -- parseInt() skips non-digits, accumulates digits, stops at the first non-digit (the '\n')
2  void setup() {
3    Serial.begin(9600);
4  }
5
6  void loop() {
7    if (Serial.available() > 0) {
8      int c = Serial.peek();                  // look without removing
9      if (c >= '0' && c <= '9') {
10       long value = Serial.parseInt();       // 25, 125, 7 ... (as a number, not characters)
11       Serial.print("got ");
12       Serial.println(value);
13     } else {
14       Serial.read();                        // discard '\n' or anything that is not a digit
```

```
    }
  }
}
```

ex17 Line protocol

```
// ex17 -- a line protocol feeding a 3-state machine. The command changes the mode; loop() runs the mode.
const int LED = LED_BUILTIN;
const unsigned long BLINK_PERIOD = 250;

enum class Mode { OFF, ON, BLINK };
Mode mode = Mode::OFF;
unsigned long lastToggle = 0;
bool ledOn = false;

void setup() {
  Serial.begin(9600);
  pinMode(LED, OUTPUT);
}

void loop() {
  // 1. input: one complete line, only when something has arrived
  if (Serial.available() > 0) {
    String line = Serial.readStringUntil('\n');
    line.trim();                                // remove '\r' and spaces
    if (line == "ON")         { mode = Mode::ON;    digitalWrite(LED, HIGH); ledOn = true; }
    else if (line == "OFF")   { mode = Mode::OFF;   digitalWrite(LED, LOW);  ledOn = false; }
    else if (line == "BLINK") { mode = Mode::BLINK; lastToggle = millis(); }
    else if (line.length() > 0) { Serial.print("unknown: "); Serial.println(line); }
  }
  // 2. behaviour of the current mode
  if (mode == Mode::BLINK && millis() - lastToggle >= BLINK_PERIOD) {
    lastToggle += BLINK_PERIOD;
    ledOn = !ledOn;
    digitalWrite(LED, ledOn);
  }
}
```

ex18 Two boards, board A

```
// ex18 board A -- production station: sends the raw byte value every 2 s
int production = 5;

void setup() {
  Serial.begin(9600);
}

void loop() {
  Serial.write(production);   // ONE byte with the numeric value (not its decimal text)
  production++;
  delay(2000);
}
```

ex18 Two boards, board B

```
// ex18 board B -- monitoring station: LED on when production >= 10
const int LED = LED_BUILTIN;
int productionReceived = 0;

void setup() {
  Serial.begin(9600);
  pinMode(LED, OUTPUT);
```

```
}

void loop() {
  if (Serial.available() > 0) {
    productionReceived = Serial.read();        // the byte A wrote: 5, 6, 7, ...
    digitalWrite(LED, productionReceived >= 10 ? HIGH : LOW);
  }
}
```

ex19 Handshake, board A

```
// ex19 board A -- sends a request value, lights its LED according to B's answer
const int LED = LED_BUILTIN;
int value = 15;

void setup() {
  Serial.begin(9600);
  pinMode(LED, OUTPUT);
}

void loop() {
  static unsigned long lastSend = 0;
  if (millis() - lastSend >= 1000) {     // one request per second, non-blocking
    lastSend = millis();
    Serial.write(value);
  }
  if (Serial.available() > 0) {          // B's answer: raw byte 1 or 0
    int reply = Serial.read();
    digitalWrite(LED, reply == 1 ? HIGH : LOW);
  }
}
```

ex19 Handshake, board B

```
// ex19 board B -- analyses the received value and acknowledges with 1 or 0
void setup() {
  Serial.begin(9600);
}

void loop() {
  if (Serial.available() > 0) {
    int received = Serial.read();
    if (received >= 10) Serial.write(1);
    else                Serial.write(0);
  }
}
```

ex20 Smart-home lighting FSM

```
// ex20 -- state-based reasoning: "where am I?" then "what does the event do here?"
const int LED = 13;
const int PB = 12;
const unsigned long BLINK_PERIOD = 500;
const unsigned long DEBOUNCE_DELAY = 50;

enum class State { OFF, ON, BLINK };
State currentState = State::OFF;
unsigned long lastBlink = 0;

// ---- debounced button (returns true once per press) ----
int buttonState = HIGH, lastReading = HIGH;
unsigned long lastDebounceTime = 0;
```

```
bool buttonPressed() {
  int reading = digitalRead(PB);
  bool pressed = false;
  if (reading != lastReading) lastDebounceTime = millis();
  if (millis() - lastDebounceTime > DEBOUNCE_DELAY && reading != buttonState) {
    buttonState = reading;
    if (buttonState == LOW) pressed = true;
  }
  lastReading = reading;
  return pressed;
}

// ---- actions ----
void turnOff() { digitalWrite(LED, LOW); }
void turnOn()  { digitalWrite(LED, HIGH); }
void blink() {
  unsigned long now = millis();
  if (now - lastBlink >= BLINK_PERIOD) {
    lastBlink = now;
    digitalWrite(LED, !digitalRead(LED));   // reading an OUTPUT pin returns what we wrote
  }
}

void setup() {
  pinMode(LED, OUTPUT);
  pinMode(PB, INPUT_PULLUP);
  turnOff();
}

void loop() {
  bool pressed = buttonPressed();
  switch (currentState) {
    case State::OFF:
      if (pressed) { turnOn(); currentState = State::ON; }
      break;
    case State::ON:
      if (pressed) { lastBlink = millis(); currentState = State::BLINK; }
      break;
    case State::BLINK:
      if (pressed) { turnOff(); currentState = State::OFF; }
      else blink();                          // "stay" action of the BLINK state
      break;
  }
}
```

ex21 FSM with timeout

```
// ex21 -- a timed state: record when it was entered, generate the TIMEOUT event from millis()
const int LED = 13;
const int PB = 12;
const unsigned long TIMEOUT = 2000;
const unsigned long BLINK_PERIOD = 200;   // 100 ms ON, 100 ms OFF
const unsigned long DEBOUNCE_DELAY = 50;

enum class State { IDLE, ACTIVE, BLINKING };
State currentState = State::IDLE;
unsigned long stateEnteredAt = 0;   // when BLINKING began
unsigned long lastBlink = 0;

int buttonState = HIGH, lastReading = HIGH;
unsigned long lastDebounceTime = 0;
bool buttonPressed() {
  int reading = digitalRead(PB);
  bool pressed = false;
  if (reading != lastReading) lastDebounceTime = millis();
  if (millis() - lastDebounceTime > DEBOUNCE_DELAY && reading != buttonState) {
    buttonState = reading;
    if (buttonState == LOW) pressed = true;
```

```
  }
  lastReading = reading;
  return pressed;
}

void setup() {
  pinMode(LED, OUTPUT);
  pinMode(PB, INPUT_PULLUP);
  digitalWrite(LED, LOW);
  Serial.begin(9600);
}

void loop() {
  bool pressed = buttonPressed();
  unsigned long now = millis();

  switch (currentState) {
    case State::IDLE:
      if (pressed) {
        digitalWrite(LED, HIGH);
        currentState = State::ACTIVE;
        Serial.println("IDLE -> ACTIVE");
      }
      break;

    case State::ACTIVE:
      if (pressed) {
        currentState = State::BLINKING;
        stateEnteredAt = now;            // start the 2 s timeout
        lastBlink = now;
        digitalWrite(LED, LOW);
        Serial.println("ACTIVE -> BLINKING");
      }
      break;

    case State::BLINKING:
      if (pressed || now - stateEnteredAt >= TIMEOUT) {   // two events lead to the same transition
        digitalWrite(LED, LOW);
        currentState = State::IDLE;
        Serial.println("BLINKING -> IDLE");
      } else if (now - lastBlink >= BLINK_PERIOD / 2) {  // stay action: blink
        lastBlink = now;
        digitalWrite(LED, !digitalRead(LED));
      }
      break;
  }
}
```

ex22 Table-driven traffic light

```
// ex22 -- table-driven FSM, non-blocking version of the lab's Listing 1
enum class State { GREEN, YELLOW, RED };
enum class Event { TIMEOUT, BLINK };

struct Transition {
  State from;
  Event event;
  State to;
  void (*action)();          // function pointer executed during the transition
};

const int ledRED = 6, ledYELLOW = 5, ledGREEN = 4;
const unsigned long RED_TIME = 5000, GREEN_TIME = 5000, YELLOW_TIME = 3000, BLINK_HALF = 100;

State currentState = State::RED;
unsigned long stateEnteredAt = 0;   // replaces delay(): when did the current state begin?
unsigned long blinkTime = 0;        // last yellow toggle

```

```
// ---- actions (entry of each state) ----
void turnOnRed() {
  digitalWrite(ledRED, HIGH); digitalWrite(ledGREEN, LOW); digitalWrite(ledYELLOW, LOW);
  stateEnteredAt = millis();
}
void turnOnGreen() {
  digitalWrite(ledRED, LOW); digitalWrite(ledGREEN, HIGH); digitalWrite(ledYELLOW, LOW);
  stateEnteredAt = millis();
}
void turnOnYellow() {
  digitalWrite(ledRED, LOW); digitalWrite(ledGREEN, LOW); digitalWrite(ledYELLOW, HIGH);
  stateEnteredAt = millis();
  blinkTime = stateEnteredAt;
}
void blinkYellow() {
  if (millis() - blinkTime >= BLINK_HALF) {
    blinkTime = millis();
    digitalWrite(ledYELLOW, !digitalRead(ledYELLOW));
  }
}

Transition transitions[] = {
  { State::RED,    Event::TIMEOUT, State::GREEN,  &turnOnGreen  },
  { State::GREEN,  Event::TIMEOUT, State::YELLOW, &turnOnYellow },
  { State::YELLOW, Event::BLINK,   State::YELLOW, &blinkYellow  },   // self-loop
  { State::YELLOW, Event::TIMEOUT, State::RED,    &turnOnRed    },
};
const int NB_TRANSITIONS = sizeof(transitions) / sizeof(transitions[0]);

// ---- engine ----
void processEvent(Event event = Event::TIMEOUT) {
  for (int i = 0; i < NB_TRANSITIONS; i++) {
    if (transitions[i].from == currentState && transitions[i].event == event) {
      transitions[i].action();
      currentState = transitions[i].to;
      break;
    }
  }
}

unsigned long durationOf(State s) {
  switch (s) {
    case State::RED:    return RED_TIME;
    case State::GREEN:  return GREEN_TIME;
    default:            return YELLOW_TIME;
  }
}

void setup() {
  pinMode(ledRED, OUTPUT); pinMode(ledYELLOW, OUTPUT); pinMode(ledGREEN, OUTPUT);
  turnOnRed();                                  // entry action of the initial state
}

void loop() {
  unsigned long now = millis();
  if (now - stateEnteredAt >= durationOf(currentState)) {
    processEvent(Event::TIMEOUT);               // the only place a timeout is generated
  } else if (currentState == State::YELLOW) {
    processEvent(Event::BLINK);                 // stay in YELLOW, toggle the LED
  }
}
```

ex23 Pedestrian request

```
// ex23 -- only the table and the event generation grow; processEvent() is untouched
enum class State { GREEN, YELLOW, RED };
enum class Event { TIMEOUT, BLINK, PED_REQUEST };

```

```
struct Transition { State from; Event event; State to; void (*action)(); };

const int ledRED = 6, ledYELLOW = 5, ledGREEN = 4;
const int PED_BUTTON = 2;
const unsigned long RED_TIME = 5000, GREEN_TIME = 5000, YELLOW_TIME = 3000, BLINK_HALF = 100;
const unsigned long DEBOUNCE_DELAY = 50;

State currentState = State::RED;
unsigned long stateEnteredAt = 0, blinkTime = 0;

void turnOnRed()    { digitalWrite(ledRED, HIGH); digitalWrite(ledGREEN, LOW); digitalWrite(ledYELLOW, LOW);
     stateEnteredAt = millis(); }
void turnOnGreen()  { digitalWrite(ledRED, LOW); digitalWrite(ledGREEN, HIGH); digitalWrite(ledYELLOW, LOW);
     stateEnteredAt = millis(); }
void turnOnYellow() { digitalWrite(ledRED, LOW); digitalWrite(ledGREEN, LOW); digitalWrite(ledYELLOW, HIGH);
     stateEnteredAt = millis(); blinkTime = stateEnteredAt; }
void blinkYellow()  { if (millis() - blinkTime >= BLINK_HALF) { blinkTime = millis(); digitalWrite(ledYELLOW, !
     digitalRead(ledYELLOW)); } }

Transition transitions[] = {
  { State::RED,    Event::TIMEOUT,     State::GREEN,  &turnOnGreen  },
  { State::GREEN,  Event::TIMEOUT,     State::YELLOW, &turnOnYellow },
  { State::YELLOW, Event::BLINK,       State::YELLOW, &blinkYellow  },
  { State::YELLOW, Event::TIMEOUT,     State::RED,    &turnOnRed    },
  { State::GREEN,  Event::PED_REQUEST, State::YELLOW, &turnOnYellow },   // new
  { State::RED,    Event::PED_REQUEST, State::GREEN,  &turnOnGreen  },   // new
};
const int NB_TRANSITIONS = sizeof(transitions) / sizeof(transitions[0]);

void processEvent(Event event = Event::TIMEOUT) {           // unchanged engine
  for (int i = 0; i < NB_TRANSITIONS; i++) {
    if (transitions[i].from == currentState && transitions[i].event == event) {
      transitions[i].action();
      currentState = transitions[i].to;
      break;
    }
  }
}

unsigned long durationOf(State s) {
  if (s == State::RED) return RED_TIME;
  if (s == State::GREEN) return GREEN_TIME;
  return YELLOW_TIME;
}

int buttonState = HIGH, lastReading = HIGH;
unsigned long lastDebounceTime = 0;
bool buttonPressed() {
  int reading = digitalRead(PED_BUTTON);
  bool pressed = false;
  if (reading != lastReading) lastDebounceTime = millis();
  if (millis() - lastDebounceTime > DEBOUNCE_DELAY && reading != buttonState) {
    buttonState = reading;
    if (buttonState == LOW) pressed = true;
  }
  lastReading = reading;
  return pressed;
}

void setup() {
  pinMode(ledRED, OUTPUT); pinMode(ledYELLOW, OUTPUT); pinMode(ledGREEN, OUTPUT);
  pinMode(PED_BUTTON, INPUT_PULLUP);
  turnOnRed();
}

void loop() {
  if (buttonPressed()) processEvent(Event::PED_REQUEST);    // external event
  unsigned long now = millis();
  if (now - stateEnteredAt >= durationOf(currentState)) processEvent(Event::TIMEOUT);
  else if (currentState == State::YELLOW) processEvent(Event::BLINK);
}
```

ex24 PIR alarm

```
// ex24 -- three timed states; the PIR is only looked at in STANDBY
const int PIR = 2;
const int BUZZER = 11;
const int GREEN = 13;
const int RED = 12;
const unsigned long ALERT_TIME = 3000, COOLDOWN_TIME = 2000, BEEP_HALF = 300;

enum class State { STANDBY, ALERT, COOLDOWN };
State currentState = State::STANDBY;
unsigned long stateEnteredAt = 0, lastBeep = 0;

void enterStandby() {
  digitalWrite(GREEN, HIGH); digitalWrite(RED, LOW); digitalWrite(BUZZER, LOW);
  currentState = State::STANDBY; stateEnteredAt = millis();
  Serial.println("State: STANDBY");
}
void enterAlert() {
  digitalWrite(GREEN, LOW); digitalWrite(RED, HIGH); digitalWrite(BUZZER, HIGH);
  currentState = State::ALERT; stateEnteredAt = millis(); lastBeep = stateEnteredAt;
  Serial.println("State: ALERT");
}
void enterCooldown() {
  digitalWrite(RED, LOW); digitalWrite(BUZZER, LOW);
  currentState = State::COOLDOWN; stateEnteredAt = millis();
  Serial.println("State: COOLDOWN");
}

void setup() {
  pinMode(PIR, INPUT);          // the sensor drives the line: no pull-up needed
  pinMode(BUZZER, OUTPUT); pinMode(GREEN, OUTPUT); pinMode(RED, OUTPUT);
  Serial.begin(9600);
  enterStandby();
}

void loop() {
  unsigned long now = millis();
  switch (currentState) {
    case State::STANDBY:
      if (digitalRead(PIR) == HIGH) enterAlert();
      break;
    case State::ALERT:
      if (now - stateEnteredAt >= ALERT_TIME) enterCooldown();
      else if (now - lastBeep >= BEEP_HALF) {           // beep: toggle the buzzer
        lastBeep = now;
        digitalWrite(BUZZER, !digitalRead(BUZZER));
      }
      break;
    case State::COOLDOWN:                               // PIR ignored here
      if (now - stateEnteredAt >= COOLDOWN_TIME) enterStandby();
      break;
  }
}
```

ex25 Industrial controller

```
// ex25 -- everything from the course in one sketch: debounce, several millis() timers, a 3-state FSM
const int BUTTON = 3;
const int GREEN = 12;
const int RED = 11;
const unsigned long BLINK = 500, PRODUCT_INTERVAL = 2000, RED_TIME = 5000, DEBOUNCE_DELAY = 50;
const int MAX_PRODUCTS = 10;

enum class State { STOPPED, RUNNING, FINISHING };
State state = State::STOPPED;

unsigned long lastBlink = 0, lastProduct = 0, stateEnteredAt = 0;
```

```
int products = 0;
bool greenOn = false;

int buttonState = HIGH, lastReading = HIGH;
unsigned long lastDebounceTime = 0;
bool buttonPressed() {
  int reading = digitalRead(BUTTON);
  bool pressed = false;
  if (reading != lastReading) lastDebounceTime = millis();
  if (millis() - lastDebounceTime > DEBOUNCE_DELAY && reading != buttonState) {
    buttonState = reading;
    if (buttonState == LOW) pressed = true;
  }
  lastReading = reading;
  return pressed;
}

void setup() {
  pinMode(BUTTON, INPUT_PULLUP);
  pinMode(GREEN, OUTPUT); pinMode(RED, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  bool pressed = buttonPressed();     // called every loop so the debouncer keeps tracking
  unsigned long now = millis();

  switch (state) {
    case State::STOPPED:
      if (pressed) {
        state = State::RUNNING;
        products = 0;
        lastBlink = lastProduct = now;
        greenOn = true; digitalWrite(GREEN, HIGH);
      }
      break;

    case State::RUNNING:
      if (now - lastBlink >= BLINK) {                // task: green blinking
        lastBlink = now; greenOn = !greenOn; digitalWrite(GREEN, greenOn);
      }
      if (now - lastProduct >= PRODUCT_INTERVAL) {    // task: production
        lastProduct = now;
        products++;
        Serial.print("Product produced : "); Serial.println(products);
        if (products >= MAX_PRODUCTS) {               // automatic stop
          state = State::FINISHING;
          stateEnteredAt = now;
          greenOn = false; digitalWrite(GREEN, LOW);
          digitalWrite(RED, HIGH);
        }
      }
      break;

    case State::FINISHING:
      if (now - stateEnteredAt >= RED_TIME) {
        digitalWrite(RED, LOW);
        state = State::STOPPED;
      }
      break;
  }
}
```

ex26 Registers

```
// ex26 -- register-level I/O. pinMode(13, OUTPUT) is DDRB |= (1<<5); digitalWrite(13, HIGH) is PORTB |= (1<<5).
// One register write sets/clears up to 8 pins at once and takes 1 clock cycle instead of ~50 for digitalWrite().
void setup() {
  DDRB |= (1 << PB5);          // PB5 (pin 13) as output
}

void loop() {
  PORTB |= (1 << PB5);         // HIGH
  delay(500);
  PORTB &= ~(1 << PB5);        // LOW   (alternative: PORTB ^= (1 << PB5) toggles)
  delay(500);
}
```

ex27 Piano

```
// ex27 -- arrays + loop; tone() runs a hardware timer so it does not block
const int KEYS[] = {2, 3, 4};
const unsigned int NOTES[] = {262, 294, 330};   // C4 D4 E4
const int NB_KEYS = sizeof(KEYS) / sizeof(KEYS[0]);
const int BUZZER = 8;

void setup() {
  for (int i = 0; i < NB_KEYS; i++) pinMode(KEYS[i], INPUT_PULLUP);
  pinMode(BUZZER, OUTPUT);
}

void loop() {
  int playing = -1;
  for (int i = 0; i < NB_KEYS; i++) {
    if (digitalRead(KEYS[i]) == LOW) { playing = i; break; }   // first (lowest pin) key held
  }
  if (playing >= 0) tone(BUZZER, NOTES[playing]);
  else noTone(BUZZER);
}
```

10 J. One-Page Reference

Board. ATmega328P, 16 MHz, 5 V. Flash 32 KB (program), SRAM 2 KB (variables), EEPROM 1 KB. D0–D13 (0/1 = Serial), PWM 3 5 6 9 10 11, A0–A5 (0–1023). 40 mA max per pin, 20 recommended. Pin 13 = PB5 = "L" LED via 1 kΩ.

Types. `int` 16-bit (±32 767), `long` 32-bit, `unsigned long` for `millis()`, `byte` 0–255, `char` = one ASCII code, `float` = `double` 32-bit and slow. Suffix big constants `UL`.

LED. pin → R → anode (long leg); cathode → GND. $I = (5 - V_F)/R$: 1 kΩ 3 mA, 330 Ω 9 mA, 220 Ω 13.6 mA, 150 Ω 20 mA.

Signal. $T = t_{ON} + t_{OFF}$, $f = 1/T$, $D = t_{ON}/T$.

Output. `pinMode(p, OUTPUT)`; `digitalWrite(p, HIGH/LOW)`; `analogWrite(p, 0..255)` = PWM 490 Hz (980 on 5, 6).

Input. Pull-down: button to 5 V, R to GND, pressed = HIGH. Pull-up: button to GND, R to 5 V, pressed = LOW. `INPUT_PULLUP`: button to GND, pressed = LOW. Never leave an input floating.

Debounce. Restart a timer on every raw change; accept the new level after 50 ms stable; report the HIGH→LOW edge once.

Time. `delay(ms)` blocks. `millis()` counts. Pattern: `if (now - prev >= T) { prev = now; ...}`. One `prev` per task. Wraps after 49.7 days; subtraction survives it.

Serial. `begin(9600)`: 1 ms per byte. `print` = text, `write` = raw byte. `available()` then `read()` (one byte, −1 if none), `peek()` (no removal), `parseInt()` (whole number), `readStringUntil('\n')`. A↔B: TX→RX crossed, GND common.

Registers. `DDRx` direction, `PORTx` output/pull-up, `PINx` read. `PORTB |= (1<<PB5)` on, `&= ~(1<<PB5)` off, `^=` toggle.

FSM. States (`enum class`), events, transitions (table or `switch`), actions. Entry action sets all outputs and `stateEnteredAt`. Timeout: `now - stateEnteredAt >= duration`. `loop()` generates events; never `delay()`; `break` after a transition; log transitions, not states.

Canvas code bugs to recognise. missing `{` after `switch`; variable declared under `case`; `turnOn` vs `turnON`; undefined constants/functions; `millis()` in an `int`; `print` where `write` was needed; level used as an event.